

Brightcove Solution Design

1. Objective

The objective is to integrate Brightcove video playback into the EDS site in a way that is easy for authors to use, scalable across multiple accounts and player configurations, and lightweight for the frontend.

The solution:

- provide a simple authoring experience in Universal Editor,
- avoid exposing Brightcove technical IDs to authors,
- support both video and playlist use cases,
- and keep the integration aligned with Edge Delivery principles.

2. Proposed Architecture

The recommended approach is to use a Brightcove block in EDS that reads authoring inputs from Universal Editor and resolves them at runtime using a central configuration spreadsheet.

2.1 Configuration Spreadsheet

A configuration spreadsheet will act as the single source of truth for all Brightcove mappings.

It will store the relationship between:

- Account names,
- Brightcove account IDs,
- player names,
- player IDs,
- player type,
- and playlist alignment.

Example structure

Account Name	Account ID	Player Name	Player ID	Player Type	Playlist Align
CBD	36632107 62001	Default Player	abc123def	default	

CBD	36632107 62001	Playlist with right rail	xyz456ghi	playlist	right-rail
CBD	36632107 62001	Playlist with bottom rail	jkl789mno	playlist	bottom- rail
PGH	66500159 1001	Default Player	pqr012stu	default	
PGH	66500159 1001	Playlist with bottom rail	9wemoeX 81	playlist	bottom- rail
PGH	66500159 1001	Playlist with right rail	vwx345yz a	playlist	right-rail
FEI	88765432 1001	Default Player	bcd678efg	default	
FEI	88765432 1001	Playlist with right rail	hij901klm	playlist	right-rail
FEI	88765432 1001	Playlist with bottom rail	nop234qr s	playlist	bottom- rail

This spreadsheet can be published as a JSON file, for example:

```
1 | /content/config/brightcove.json
```

At runtime, the block fetches this file and resolves the values selected by the author into the correct Brightcove IDs.

3. Authoring Experience

In Universal Editor, authors will select values from a dialog

For the video component

The author selects:

- account name – dropdown
- video ID --textfield
- call to Action

For example:

- Account = CBD
- Video ID = 6293625135001

The block then resolves:

- Account Name → Account ID
- default player for that account
- Video ID → Brightcove asset

For the playlist component

The author selects:

- account name --- dropdown
- player type --- dropdown
- playlist ID --- textfield
- call to Action

For example:

- Account = PGH
- Player Type = bottom-rail
- Playlist ID = 12345

The block then resolves the corresponding Brightcove player configuration and renders the playlist player with the selected layout.

Authoring principle

Authors will see names such as:

- CBD
- PGH
- FEI
- Default Player
- Playlist with bottom rail
- Playlist with right rail

They should not need to know:

- Brightcove account IDs,
- Brightcove player IDs,
- or script-level implementation details.

4. What the Block Will Do

The Brightcove block will be responsible for the runtime rendering and embedding of the Brightcove player.

Responsibilities of the block

- Read authored values from the DOM.
- Fetch the Brightcove configuration JSON.
- Resolve friendly names to Brightcove account and player IDs.
- Inject the Brightcove player script dynamically.
- Render the correct Brightcove video or playlist player.
- Optionally lazy-load the player to protect performance and Lighthouse scores.

Runtime rendering example

For a video embed, the block will render markup similar to:

```
1 <video
2   data-video-id="VIDEO_ID"
3   data-account="ACCOUNT_ID"
4   data-player="PLAYER_ID"
5   data-embed="default"
6   class="video-js"
7   controls
8 ></video>
9 <script
  src="https://players.brightcove.net/ACCOUNT_ID/PLAYER_ID_default/index.
  min.js"></script>
```

Brightcove player behavior

The Brightcove player JavaScript is loaded from Brightcove's CDN, typically from:

```
1 https://players.brightcove.net/ACCOUNT_ID/PLAYER_ID_default/index.min.js
```

Once loaded, the player talks directly to Brightcove's backend using the configured account, player, and video or playlist IDs.

This means:

- no extra AEM backend integration is required,

- no EDS server-side processing is needed,
- and the Brightcove embed remains fully managed by the frontend block.

4.1 [Schema.org - Schema.org](#) (JSON-LD) Integration

As part of SEO and LLM requirements, every page containing a Brightcove video or playlist should expose  [Schema.org - Schema.org](#) **metadata** (VideoObject) in JSON-LD format.

This ensures:

- better search engine visibility
- improved content discoverability
- compatibility with LLM-based consumption

Approach Options

Two approaches can be considered for implementing JSON-LD in EDS:

Option 1: Manual JSON-LD via Page Metadata

Edge Delivery supports adding JSON-LD through page metadata.

In this approach:

- Authors manually provide a JSON-LD (VideoObject) snippet in the page metadata.

Pros

- Fully compatible with native EDS JSON-LD handling

Cons

- Manual effort for authors

Option 2: Automated JSON-LD via Edge Worker

Since EDS has no traditional server-side rendering, JSON-LD can be dynamically injected at the **edge layer**.

In this approach:

- An Edge Worker intercepts the HTML response
- Detects Brightcove video/playlist blocks
- Fetches metadata from the Brightcove Playback API
- Injects  [Schema.org - Schema.org](#) JSON-LD into the page

High-Level Flow

1. Author publishes page with Video/Playlist block

2. AEM generates HTML with authored values (account, videoId/playlistId)
3. HTML is served via Edge Delivery CDN
4. Edge Worker:
 - Parses HTML to detect Brightcove blocks
 - Extracts account and video/playlist ID
 - Resolves account ID using configuration
 - Calls Brightcove Playback API
 - Constructs VideoObject JSON-LD
 - Injects `<script type="application/ld+json">` into HTML
5. Final response is delivered to the browser with  [Schema.org - Schema.org](https://schema.org) metadata

Sample JSON-LD

```
1 <script type="application/ld+json">
2 {
3   "@context": "https://schema.org",
4   "@type": "VideoObject",
5   "name": "Video Title",
6   "description": "Video description",
7   "thumbnailUrl": ["https://.../image.jpg"],
8   "uploadDate": "2022-01-25T22:55:01.489Z",
9   "duration": "PT1M9S",
10  "contentUrl": "https://.../video.mp4",
11  "embedUrl":
12    "https://players.brightcove.net/{accountId}/{playerId}_default/index.html?videoId={videoId}"
13 }
14 </script>
```

5. Advantages of the Approach

This architecture provides several benefits:

- **Simple authoring** through Universal Editor
- **Centralized configuration** through one maintained spreadsheet/JSON source
- **No exposure of technical IDs** to authors
- **Scalable support** for multiple accounts and player variations
- **Low implementation complexity** compared to a full catalog integration
- **Frontend-driven rendering** that fits the EDS model
- **Better governance** because mappings can be controlled centrally
- **Performance flexibility** through lazy loading and controlled script injection

6. Other Viable Authoring Options

There are three possible authoring models for this integration.

Option 1: Simple authoring with account name + video ID

This is the lightest implementation.

The author selects:

- account name
- video ID
- call to Action

The block resolves the account and default player using the configuration spreadsheet.

This is best when authors already know the Brightcove video ID .

Option 2: Paste Brightcove URL and parse IDs in code

In this model, the author pastes a Brightcove share URL into a single field.

The block parses the URL and extracts the required Brightcove identifiers at runtime.

This improves ease of use for authors compared to typing IDs manually, while still keeping the implementation relatively simple.

Option 3: Full Brightcove catalog integration through UI customization

A custom UI can be built using:

- UI extensibility,
- App Builder extension,
- or a custom data type in Universal Editor.

This custom experience would:

- connect to Brightcove CMS or Playback APIs,
- support video search and selection,
- return catalog results to the authoring interface,
- and allow authors to choose content directly from the Brightcove library.

It requires:

- additional custom development,
- authentication handling,
- API integration